GOONzionario

Created	@January 26, 2026 9:21 PM
Class	programmazione a oggetti

Formulario Pratico di Java per l'Esame di Programmazione

Template Generici per Classi, Collezioni, Generics e Iteratori

1. Struttura della Classe e Information Hiding

1.1 Template Classe Base (Esercizio 1)

Usa questo schema per qualsiasi entità che debba memorizzare dati.

- **Campi**: `private final` per dati immutabili, `private` per quelli variabili.
- **Costruttore**: Inserisci sempre i controlli di validità (Precondizioni).

```
import java.util.Objects;

public class EntitaBase {
    private final String id;      // Immutabile
    private double valore;        // Mutabile

    public EntitaBase(String id, double valore) {
        // Validazione standard
        if (id == null || id.isEmpty()) throw new IllegalArgumentException("ID null o o vuoto");
        if (valore < 0) throw new IllegalArgumentException("Valore negativo non ammesso");

        this.id = id;
```

```
        this.valore = valore;
    }

    public String getId() { return id; }
    public double getValore() { return valore; }

    public void setValore(double valore) {
        if (valore >= 0) this.valore = valore; // Validazione anche nel setter
    }
}
```

1.2 Template Sottoclasse (Ereditarietà)

Usa questo schema quando la traccia dice "Classe B sottotipo di A" o "B estende A".

```
public class SottoClasse extends SuperClasse {
    private final String attributoExtra;

    public SottoClasse(String campoPadre1, int campoPadre2, String extra) {
        super(campoPadre1, campoPadre2); // Obbligatorio come prima istruzion
e
        if (extra == null) throw new IllegalArgumentException();
        this.attributoExtra = extra;
    }

    // Override di metodi della superclasse se necessario
}
```

2. Metodi Standard (Object) - Template Universale

Copia e incolla questo schema per `equals` e `hashCode`. È il modo più sicuro per non sbagliare i confronti, specialmente con i numeri decimali.

```
@Override
public boolean equals(Object obj) {
    // 1. Stesso riferimento?
    if (this == obj) return true;
    // 2. Null o classi diverse? (Usa getClass() per precisione)
    if (obj == null || this.getClass() != obj.getClass()) return false;

    NomeClasse other = (NomeClasse) obj;

    // 3. Confronto campi (String/Oggetti → Objects.equals, Primitivi → ==)
    return Objects.equals(this.campoString, other.campoString) &&
           this.campoInt == other.campoInt &&
           Double.compare(this.campoDouble, other.campoDouble) == 0;
}

@Override
public int hashCode() {
    // Deve usare gli stessi campi usati in equals
    return Objects.hash(campoString, campoInt, campoDouble);
}
```

3. Aggregatori e Collezioni (Esercizio 2)

3.1 Gestione Sicura di Liste/Set

Quando una classe "contiene" una collezione di altri oggetti.

```
import java.util.*;

public class Aggregatore {
    private final List<Elemento> elementi; // final: il riferimento alla lista non ca
mbia

    public Aggregatore(Collection<Elemento> input) {
        if (input == null) throw new IllegalArgumentException();
```

```
        // Defensive copy: creiamo una nuova lista per non dipendere da quella e
sterna
        this.elementi = new ArrayList<>(input);
    }

    // Esempio calcolo aggregato (Somma o Media)
    public double calcolaTotale() {
        double acc = 0;
        for (Elemento e : elementi) {
            acc += e.getValore(); // Supponendo che Elemento abbia getValore()
        }
        return acc;
    }
}
```

4. Generics e Wildcards (Esercizio 3 - Logica Astratta)

4.1 Regola PECS (Producer Extends, Consumer Super)

Usa queste firme quando devi manipolare collezioni di tipi generici.

- **Leggere** da una collezione: `Collection<? extends T>` (posso estrarre T).
- **Scrivere** in una collezione: `Collection<? super T>` (posso inserire T).

```
// Esempio Metodo Generico: sposta elementi che soddisfano un predicato
public static <T> void sposta(Collection<? extends T> sorgente,
                              Collection<? super T> destinazione,
                              Predicate<? super T> filtro) {
    for (T elem : sorgente) {
        if (filtro.test(elem)) {
            destinazione.add(elem);
        }
    }
}
```

5. Iteratori con Caching (Esercizio Sequenze)

5.1 Template Sequenza con Memoria (Lazy Loading)

Schema per generare valori "al volo" ma ricordandoli per risparmiare calcoli.

```
import java.util.*;

public class GeneratoreSequenza implements Iterable<Long> {
    private final int limite;
    // La cache memorizza: Indice → Valore calcolato
    private final Map<Integer, Long> cache = new HashMap<>();

    public GeneratoreSequenza(int n) { this.limite = n; }

    @Override
    public Iterator<Long> iterator() {
        return new Iterator<Long>() {
            private int index = 0;

            @Override
            public boolean hasNext() { return index < limite; }

            @Override
            public Long next() {
                if (!hasNext()) throw new NoSuchElementException();

                // Logica Caching: se non c'è, calcola e salva
                if (!cache.containsKey(index)) {
                    cache.put(index, logicaCalcoloComplessa(index));
                }

                long result = cache.get(index);
                index++;
                return result;
            }
```

```
            private long logicaCalcoloComplessa(int i) {
                // Sostituisci con Fibonacci, Fattoriale o logica della traccia
                if (i <= 1) return i;
                return (long) i * i;
            }
        };
    }
}
```

6. Snippet di Ordine Superiore (Map / Transform)

6.1 Da Iterable a Collection (Eager)

```
public static <A, B> Collection<B> trasforma(Iterable<A> sorgente, Function
<? super A, ? extends B> f) {
    Collection<B> res = new ArrayList<>();
    for (A elem : sorgente) res.add(f.apply(elem));
    return res;
}
```

6.2 Da Iterator a Iterator (Lazy)

```
public static <A, B> Iterator<B> trasformaLazy(Iterator<A> it, Function<? supe
r A, ? extends B> f) {
    return new Iterator<B>() {
        public boolean hasNext() { return it.hasNext(); }
        public B next() { return f.apply(it.next()); }
    };
}
```

7. Prontuario Eccezioni

- `throw new IllegalArgumentException("msg");` → Parametri non validi.

- `throw new NoSuchElementException();` → `next()` chiamato su iteratore vuoto.
- `throw new IllegalStateException("msg");` → Operazione non permessa nello stato corrente.

8. Riepilogo firme Interfacce Funzionali (Java 8+)

- `Function<T, R>` → `R apply(T t)` (Trasforma T in R)
- `Predicate<T>` → `boolean test(T t)` (Verifica T)
- `Consumer<T>` → `void accept(T t)` (Consuma T)
- `Supplier<T>` → `T get()` (Genera T)

9. Stream API e Lambda (Uso Pratico)

```
import java.util.List;
import java.util.stream.Collectors;

public class EsempioStream {
    public static void main(String[] args) {
        List<Integer> nums = List.of(1, 2, 3, 4, 5);

        nums.stream()
            .filter(n -> n % 2 == 0) // Predicate
            .map(n -> n * 2)         // Function
            .forEach(System.out::println); // Consumer

        List<Integer> pari = nums.stream()
                                 .filter(n -> n % 2 == 0)
                                 .collect(Collectors.toList());
    }
}
```

10. Classi Anonime e Lambda Expression

10.1 Classi Anonime

Si usano quando serve mantenere uno stato interno (es. negli Iteratori).

```
Collections.sort(lista, new Comparator<String>() {
    @Override
    public int compare(String s1, String s2) {
        return Integer.compare(s1.length(), s2.length());
    }
});
```

10.2 Lambda Expression

Solo per interfacce funzionali (1 solo metodo).

```
lista.sort((s1, s2) → Integer.compare(s1.length(), s2.length()));
```

11. HashMap: Gestione Coppie Chiave-Valore

Indispensabile per contare occorrenze, raggruppare dati o gestire dazi/nazioni.

11.1 Metodi Essenziali

```
Map<String, Double> mappa = new HashMap<>();

// Inserimento o aggiornamento
mappa.put("Italia", 0.22);

// Recupero SICURO (evita null)
double d = mappa.getOrDefault("Francia", 0.0);

// Controllo esistenza chiave
if (mappa.containsKey("Germania")) { ... }

// Calcolo dinamico (es. caching o raggruppamento)
mappa.computeIfAbsent("Spagna", k → 0.10); // Aggiunge se manca
```

11.2 Iterazione (Il metodo migliore)

Usa `entrySet()` se ti servono sia la chiave che il valore.

```
for (Map.Entry<String, Double> entry : mappa.entrySet()) {
    String chiave = entry.getKey();
    Double valore = entry.getValue();
    System.out.println(chiave + " : " + valore);
}

// Oppure tramite forEach con Lambda
mappa.forEach((k, v) -> System.out.println(k + " -> " + v));
```